

M1 MIAGE

Université Paris-Dauphine PSL

2024-2025

Rapport Projet Java « Gestion de vente »

Asma Ben-Zine

Introduction

Contexte

Objectifs:

Description du Projet

Explication du thème du projet

Fonctionnalités

Structure du Code

Diagramme de classe:

Le diagramme relationnel de la BD:

Le diagramme des cas d'utilisation :

Méthodologie

Technologies et Outils Utilisés

Implémentation

Détails Techniques

Tests JUNIT:

Conclusion:

Introduction

Contexte

Le projet "Gestion de Vente" s'inscrit dans le cadre de l'UE Java-Objet pour le cursus de Master 1 Miage de l'année académique 2024-2025. Il vise à développer une application permettant aux utilisateurs de parcourir un catalogue de bijoux (bagues et colliers), d'effectuer des recherches avancées, de passer des commandes et de gérer les factures associées. L'application est conçue en Java avec une interface graphique utilisant Swing et une base de données MySQL.

Objectifs:

- Offrir une expérience utilisateur intuitive pour l'achat de bijoux en ligne.
- Implémenter un système de gestion de commandes et de factures.
- Permettre une recherche avancée dans le catalogue de produits.
- Assurer un accès administrateur pour la gestion des stocks, des clients, des produits et des factures.

Description du Projet

Explication du thème du projet

L'application de gestion de vente de bijoux est conçue pour répondre aux besoins des clients et des administrateurs à travers une interface intuitive et ergonomique. Elle se divise en deux parties principales, chacune ayant des fonctionnalités adaptées à son type d'utilisateur.

- **Interface Client** : Cette partie permet aux utilisateurs de naviguer librement dans le catalogue de bijoux (bagues et colliers), de rechercher des produits à l'aide de filtres avancés, et d'ajouter des articles à leur panier avant même de se connecter. Une fois authentifiés, ils peuvent finaliser leur commande, recevoir une facture automatique et suivre l'évolution de leur commande grâce aux statuts (en cours, validée, livrée). L'expérience utilisateur a été optimisée pour rendre l'achat fluide et agréable, avec la possibilité de conserver le panier en cas de connexion ultérieure.
- **Interface Administrateur** : Destinée aux responsables de la boutique, cette interface offre un contrôle total sur la gestion du catalogue, des commandes et des factures. L'administrateur peut ajouter, modifier ou supprimer des produits tout en gérant efficacement les stocks. Il a également la possibilité de superviser les commandes des clients, de valider ou modifier des factures, et d'envoyer les mises à jour aux clients. Cette interface garantit une gestion efficace et optimisée du commerce électronique, avec une visibilité complète sur les transactions et l'état des ventes.

Fonctionnalités

L'application de gestion de vente de bijoux propose une gamme complète de fonctionnalités permettant aux clients de naviguer efficacement dans le catalogue, de gérer leurs achats et de suivre leurs commandes, tout en offrant aux administrateurs des outils de gestion avancés. Grâce à une interface intuitive, elle assure une expérience fluide et optimisée pour chaque utilisateur.

1. **Catalogue de bijoux** : Affichage des bagues et colliers avec images, descriptions et stocks disponibles pour une meilleure visualisation des produits.
2. **Panier d'achat** : Possibilité d'ajouter des bijoux au panier, de modifier les quantités ou de supprimer des articles avant la validation de la commande.
3. **Gestion des commandes** : Suivi des commandes avec un état évolutif : "En cours", "Validée", "Livree", permettant aux clients de suivre l'avancement de leurs achats.

4. **Facturation** : Génération automatique d'une facture après validation d'une commande, avec possibilité de téléchargement et d'envoi par email.
5. **Gestion des clients** : Ajout et modification des informations des clients pour assurer un suivi personnalisé.
6. **Gestion des stocks** : Mise à jour des quantités après validation des commandes, suppression des produits en cas d'indisponibilité et ajustements en cas de retour client.
7. **Filtrage dynamique** Certains filtres du catalogue, comme le matériau, sont mis à jour automatiquement en fonction des bijoux disponibles dans la base de données, garantissant une recherche pertinente et actualisée.
8. **Recherche rapide pour les administrateurs**
Un bouton **Quick Search** permet aux administrateurs d'accéder rapidement aux informations essentielles. En saisissant une donnée clé (ID client, email, ID commande, ID facture), l'interface affiche immédiatement le client concerné, ainsi que l'historique de ses commandes et factures. L'administrateur peut alors consulter ou modifier ces informations en toute simplicité.

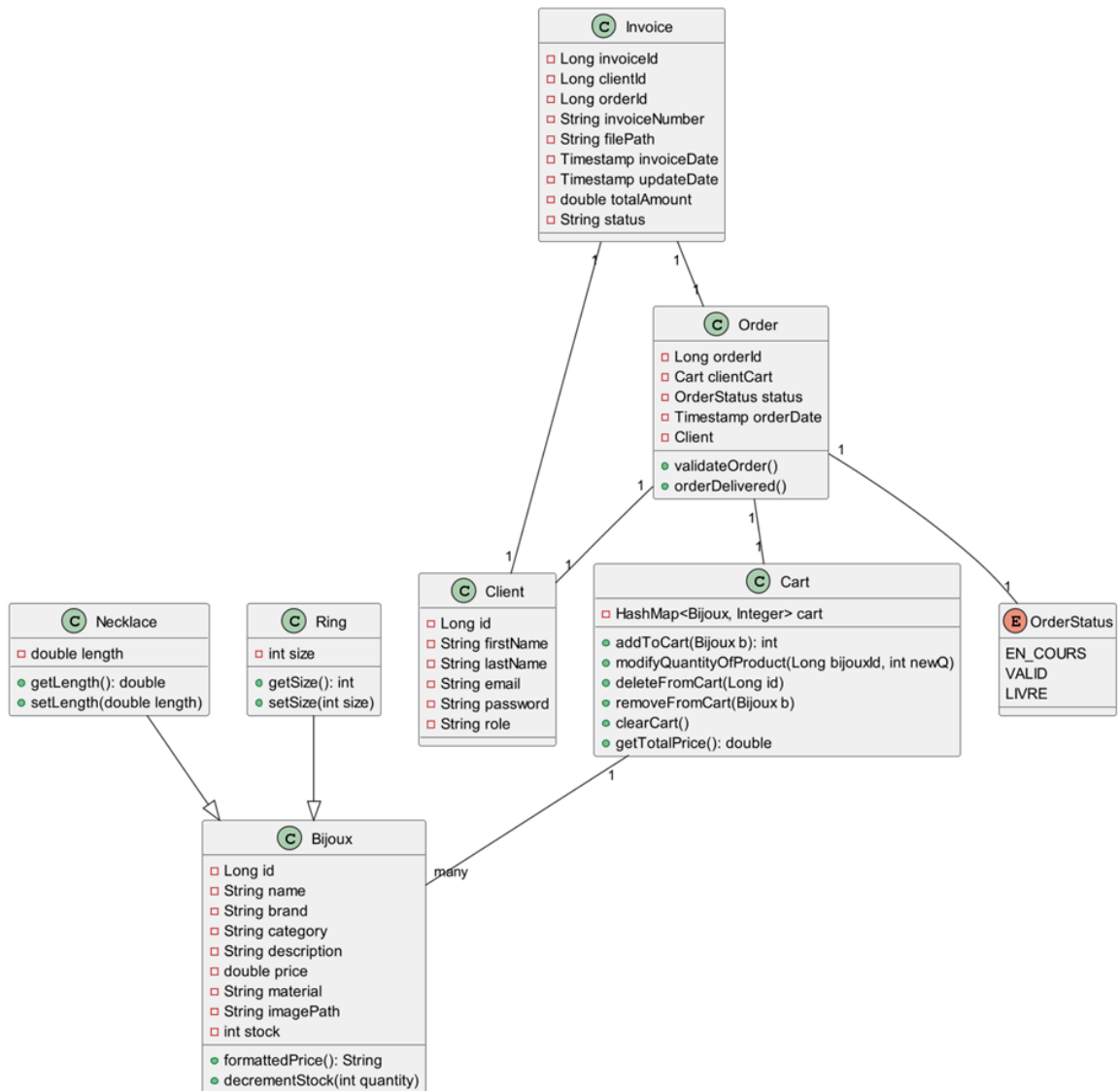
Structure du Code

L'application est structurée selon le modèle MVC (Modèle-Vue-Contrôleur) :

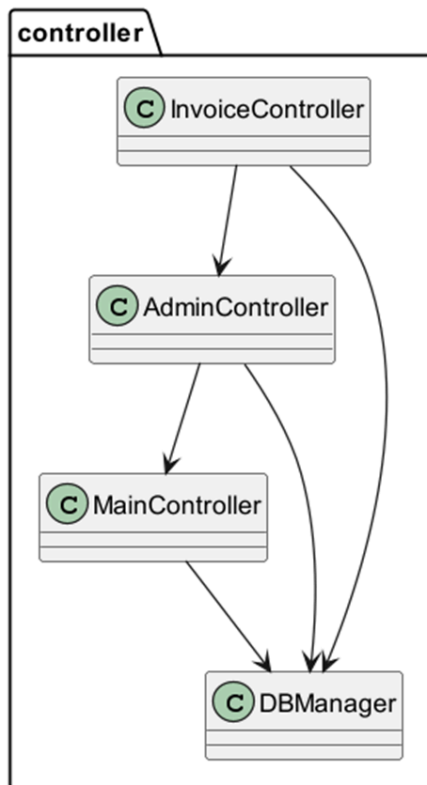
- **Modèle** : classes représentant les entités (Bijoux, Client, Commande, Facture).
- **Vue** : interface utilisateur avec Swing (catalogue, panier, administration).
- **Contrôleur** : gestion de la logique de l'application (interaction entre modèle et vue).

Diagramme de classe:

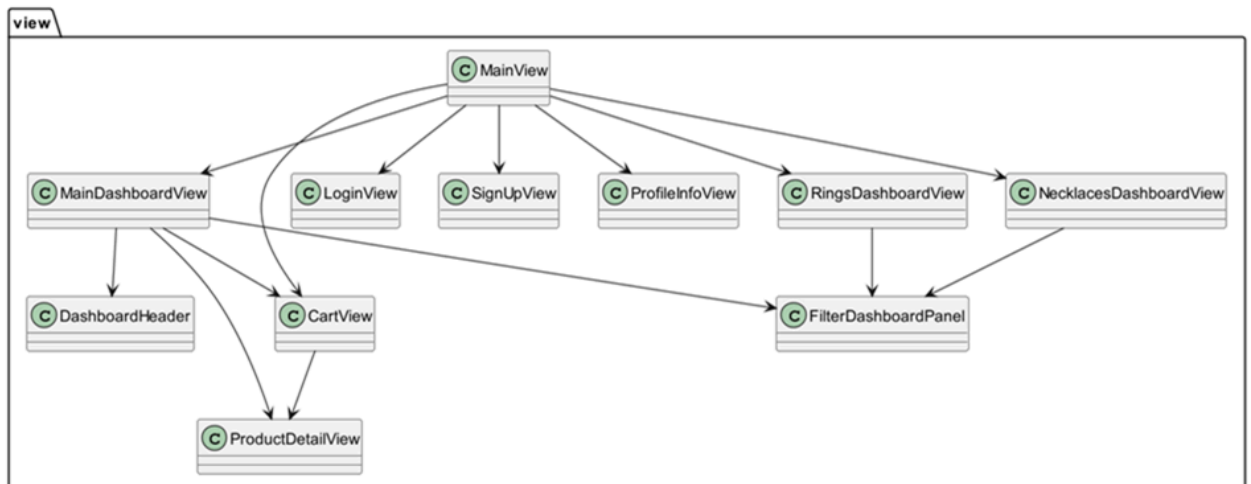
- Modèle :



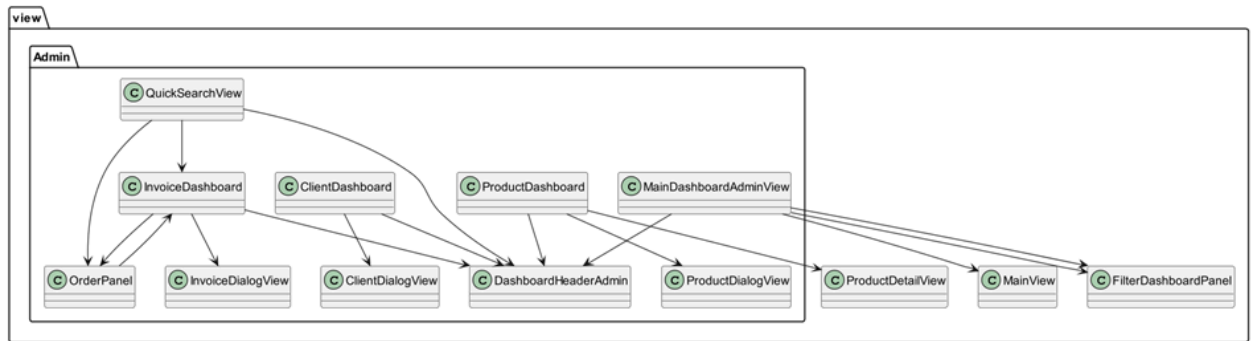
- **Contrôleur :**



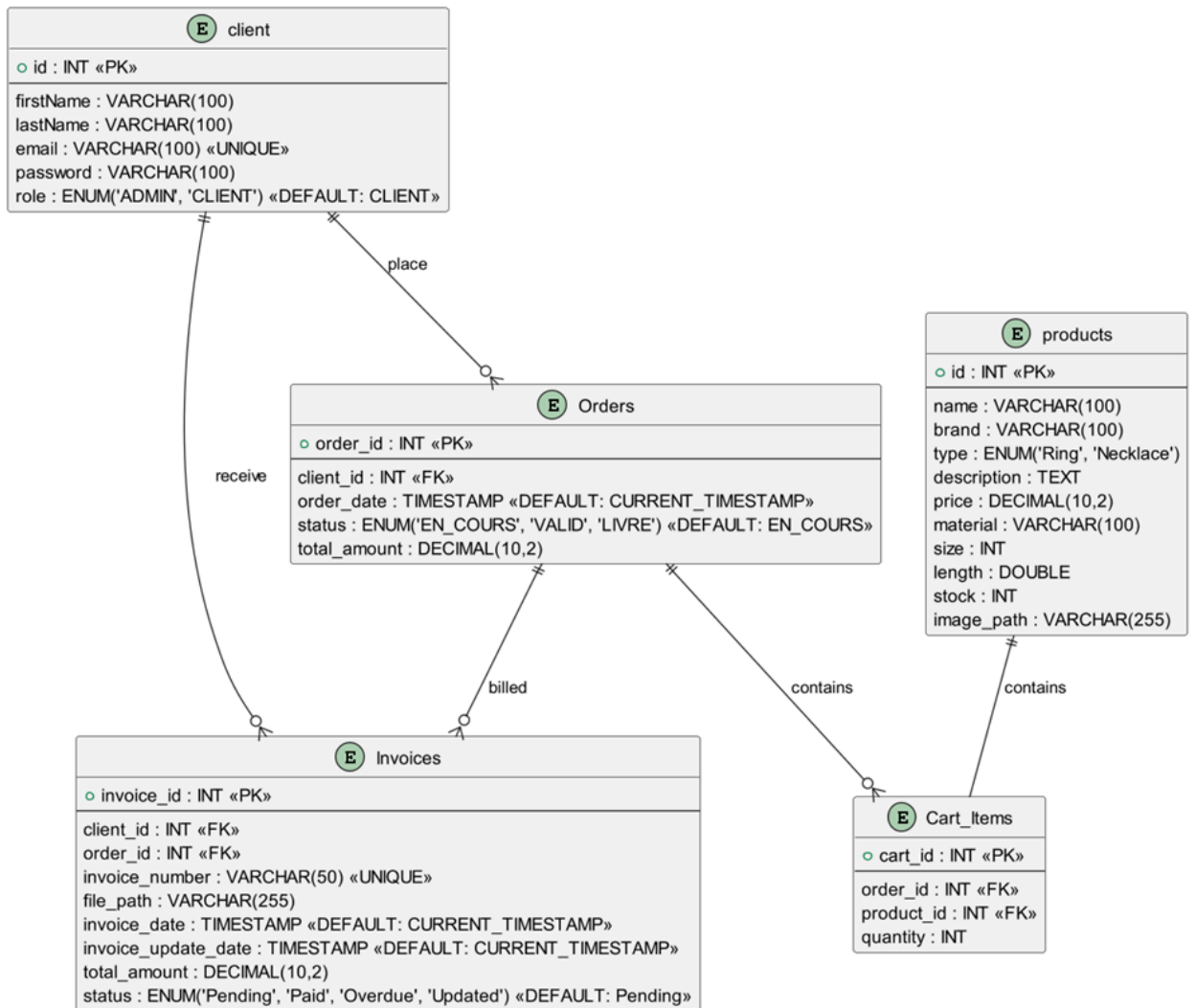
- **Vue (pour le client)**



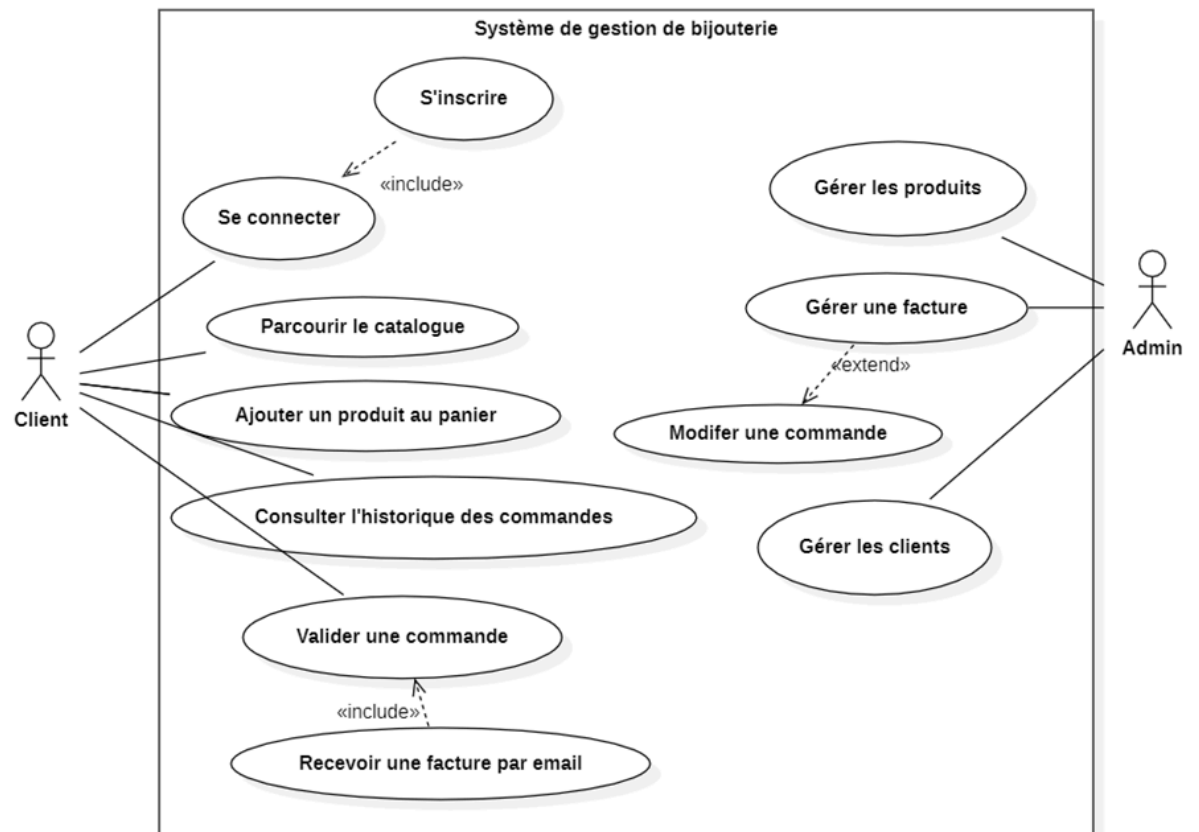
- **Vue** (pour l'administrateur)



Le diagramme relationnel de la BD:



Le diagramme des cas d'utilisation :



Méthodologie

La méthodologie suivie repose sur une approche itérative avec des tests unitaires JUnit pour assurer la qualité du code.

Technologies et Outils Utilisés

Le projet utilise différentes technologies et outils pour le développement, la gestion de version, les tests et la documentation.

- **Langage** : Java 18
- **Interface Graphique** : Swing
- **Base de données** : MySQL avec JDBC
- **Génération de factures** : iText PDF
- **Gestion de projet** : Maven
- **Tests** : JUnit 5.

Implémentation

Détails Techniques

L'architecture technique du projet repose sur une base de données MySQL, qui assure le stockage des informations relatives aux produits, clients et commandes. La communication entre l'application et la base de données est réalisée via JDBC.

Un script SQL est conservé dans les ressources du projet et permet d'initialiser les tables de la base de données au lancement de l'application.

En ce qui concerne la facturation, l'application génère automatiquement des factures au format PDF grâce à la bibliothèque iText. Ces factures peuvent être téléchargées par l'utilisateur et envoyées par email, offrant ainsi une solution complète et automatisée pour la gestion des paiements et du suivi des commandes.

Tests JUNIT:

Nous avons implémenté des tests JUnit pour plusieurs classes clés du projet, notamment **Cart**, **Order**, **Invoice**, et **MainController**, afin de vérifier la robustesse et la fiabilité des fonctionnalités essentielles.

- **CartTest** : vérifie l'ajout et la suppression de produits dans le panier.

- **OrderTest** : vérifie la création d'une commande et l'affectation du statut.
- **InvoiceTest** : test de la génération de factures et validation des informations.

Ces tests garantissent que les principales fonctionnalités du projet fonctionnent comme attendu et assurent une meilleure stabilité du système en identifiant d'éventuelles erreurs dès la phase de développement.

Conclusion:

Ce projet de gestion de vente de bijoux a été conçu pour offrir une expérience fluide et intuitive, aussi bien pour les clients que pour les administrateurs. Les clients peuvent facilement parcourir le catalogue, ajouter des produits à leur panier et passer commande, tandis que les administrateurs disposent d'outils efficaces pour gérer les stocks, les clients et les factures.

Grâce à l'utilisation de Java, MySQL, JDBC et iText, l'application assure une gestion optimisée des données et une génération automatisée des factures.

Ce projet pose une base solide pour une boutique en ligne, avec des perspectives d'amélioration intéressantes : l'intégration d'un système de paiement en ligne, l'enrichissement du catalogue avec de nouvelles catégories de produits, ou encore une optimisation de l'expérience utilisateur.